TALLER DE INTEGRACIÓN

FACULTAD: UADER FCYT

GRUPO: ROMERO GONZALO, SANGALLI LUCIANO, ZARAGOZA NAHUEL (GRUPO H)

PROFESOR ESP. JULIÁN ESCALANTE, LIC. DANIEL FIGUEROA, AUS. GABRIEL PERERO

AVANCE HASTA EL 29/10/26

---

1. Objetivo del informe

Este informe documenta el inicio de la Fase 3 del proyecto, correspondiente a la construcción del sistema. Cubre el cierre del Sprint 1, dedicado a la preparación del entorno, y los primeros avances del Sprint 2, enfocado en los primeros módulos CRUD y en el catálogo público.

Las fases anteriores de relevamiento y diseño ya fueron entregadas y corregidas, y sus definiciones son las que guían las decisiones técnicas que se describen más adelante.

2. Situación dentro del cronograma

El cronograma aprobado ubica el trabajo de esta instancia entre dos sprints:

- **Sprint 1, setup y entorno base:** del 15 al 28 de octubre. Configuración del repositorio, la base de datos relacional y el modelado de entidades. Se da por cerrado.
- **Sprint 2, primeros CRUD y catálogo:** del 29 de octubre al 11 de noviembre. Implementación del CRUD de productos con etiquetas, el CRUD de usuarios y clientes, y el catálogo público con buscador y filtros. En curso.

Al momento de este informe está operativo el entorno completo y funcionando el primero de los cinco módulos CRUD comprometidos en el backlog.

3. Trabajo realizado

3.1. Configuración del entorno

Se dejó armada la estructura del repositorio separando el servidor y el cliente en dos proyectos independientes, cada uno con sus propias dependencias. Se configuró el archivo de variables de entorno con un ejemplo versionado, de modo que cada integrante pueda levantar el proyecto con sus credenciales locales sin exponer datos sensibles en el repositorio.

La base de datos relacional quedó creada en PostgreSQL y conectada mediante Prisma, que genera el cliente de acceso a partir del esquema y permite propagar los cambios del modelo sin escribir instrucciones de creación de tablas a mano.

3.2. Modelo de datos

Se trasladó al código la porción del diagrama de clases que corresponde al catálogo y al inventario. Las entidades implementadas son Usuario, Categoría, Producto, Etiqueta, ProductoEtiqueta y MovimientoStock, junto con los tipos enumerados de rol de usuario, tipo de etiqueta y tipo de movimiento.

Las entidades restantes del modelo, como Pedido, Pago, Comprobante, Envío, Cupón y Reseña, se incorporan en los sprints que las tienen asignadas. Se optó por no adelantarlas para evitar mantener tablas sin uso durante semanas.

3.3. Interfaz de programación del catálogo

Quedó implementado el primero de los cinco CRUD previstos. La interfaz expone las operaciones de listado, consulta individual, alta, modificación y baja lógica de productos.

El listado admite búsqueda por nombre, filtrado por categoría, filtrado por una o varias etiquetas de forma simultánea, filtro por disponibilidad y paginación. Cuando se envían varias etiquetas, el sistema devuelve únicamente los productos que cumplen con todas, no con alguna de ellas, que es el comportamiento que requiere el asistente de búsqueda.

La validación de los datos de entrada se resuelve con esquemas declarativos que se ejecutan antes de tocar la base de datos. Si un dato no cumple el formato esperado, la operación se rechaza con un mensaje que indica el campo exacto y el motivo, y no se escribe nada.

3.4. Motor de inventario por eventos

Se implementó la regla de negocio central del proyecto. La entidad Producto no tiene ningún campo que guarde la cantidad disponible. La existencia de cada producto se calcula sumando los registros de movimientos asociados, donde las entradas se almacenan con cantidad positiva y las salidas con cantidad negativa.

Cada movimiento guarda la fecha, el tipo, la cantidad, una justificación obligatoria y el usuario que lo registró. Los movimientos no se editan ni se eliminan: una corrección se realiza registrando un movimiento compensatorio, de modo que el historial siempre explica cómo se llegó a la cantidad actual.

Para el listado del catálogo se resolvió el cálculo mediante una única consulta agrupada que obtiene el stock de todos los productos de la página, en lugar de una consulta por cada fila. Era necesario para que el rendimiento no se degrade a medida que crezca el catálogo.

3.5. Catálogo público

Se desarrolló la pantalla del catálogo consumiendo la interfaz del servidor. Muestra las tarjetas de producto con sus etiquetas, el precio formateado en pesos argentinos y el estado de disponibilidad en tres niveles: disponible, últimas unidades y sin stock.

Cuando un producto está agotado, el botón de compra se reemplaza por el de aviso de reposición, tal como define el caso de uso correspondiente. El buscador aplica una

espera breve entre la escritura y la consulta, para no generar una petición al servidor por cada tecla presionada.

La paleta de colores quedó configurada a partir del logo entregado por los clientes, de forma que el resto de las pantallas la hereden sin tener que repetir los valores.

3.6. Datos de prueba

Se armó un script que carga un conjunto de datos de ejemplo para poder trabajar sin depender de que los clientes entreguen el catálogo definitivo. Incluye un usuario administrador, cuatro categorías, ocho etiquetas y cuatro productos.

Los productos de prueba fueron elegidos para cubrir los tres estados de disponibilidad: uno con stock holgado, uno por debajo del mínimo configurado, uno agotado y uno en condiciones normales. Esto permite verificar el comportamiento de la interfaz sin modificar datos a mano.

El stock inicial de estos productos no se carga escribiendo un número, sino registrando movimientos de entrada, de modo que el script respeta la misma regla que el sistema en producción.

4. Estructura del proyecto

El repositorio quedó organizado en dos carpetas principales. El servidor sigue una separación por responsabilidades:

- **prisma:** define el esquema de datos y el script de carga de datos de prueba.
- **src/app.js:** configuración del servidor web y registro de las rutas.
- **src/server.js:** punto de entrada que levanta el servicio.
- **src/lib:** instancia única del cliente de base de datos.
- **src/schemas:** esquemas de validación de los datos que llegan desde el cliente.
- **src/services:** reglas de negocio independientes del transporte, como el cálculo de stock.
- **src/controllers:** lógica de cada operación de la interfaz.
- **src/routes:** definición de las direcciones y sus métodos.
- **src/middlewares:** manejo centralizado de errores.

El cliente mantiene una separación equivalente entre el acceso a la interfaz del servidor, los componentes reutilizables y las pantallas completas.

Esta organización se eligió para que las reglas de negocio queden aisladas del resto. El cálculo de stock, por ejemplo, vive en un único archivo y puede probarse por separado, sin levantar el servidor web.

5. Decisiones técnicas y su justificación

5.1. El stock no se almacena como un campo

Es la decisión más importante del modelo y responde a un requisito explícito del cliente. Al no existir un campo editable, no hay forma de que el administrador modifique la cantidad de manera arbitraria: solo puede registrar entradas y salidas, cada una con su justificación y su responsable.

El costo de esta decisión es que cada consulta del catálogo implica un cálculo adicional. Se mitigó resolviendo ese cálculo en una sola consulta agrupada por página. Si en el futuro el volumen lo justifica, se puede incorporar una vista materializada sin alterar el resto del código, porque el cálculo está encapsulado en un único servicio.

5.2. Un producto sin etiquetas no se publica

La validación exige al menos una etiqueta al dar de alta un producto. La razón es que el asistente de búsqueda filtra por especie y edad, de modo que un producto sin etiquetar quedaría invisible en la tienda aunque tuviera stock y precio cargados. Preferimos que el sistema lo impida en el momento del alta antes que descubrirlo cuando un producto no se vende.

5.3. La baja de productos es lógica, no física

Al dar de baja un producto el sistema lo marca como inactivo en lugar de eliminarlo. De esta forma deja de aparecer en la tienda pero se conserva su historial de ventas y sus movimientos de inventario, que son datos que el negocio necesita para sus informes.

5.4. Validación antes de la persistencia

Toda la información que llega desde el cliente pasa por un esquema de validación antes de alcanzar la base de datos. Esto centraliza las reglas de formato en un solo lugar y permite devolver mensajes de error específicos por campo, en lugar de un error genérico.

6. Puesta en marcha

El proyecto se levanta con los siguientes pasos, partiendo de una instalación de PostgreSQL en funcionamiento.

6.1. Base de datos

```
psql -U postgres -c "CREATE DATABASE petmarket;"
```

6.2. Servidor

```
cd backend
npm install
cp .env.example .env
npx prisma generate
npm run db:push
npm run db:seed
npm run dev
```

El archivo de variables de entorno debe completarse con la contraseña local de PostgreSQL antes de ejecutar los comandos de base de datos. El servicio queda escuchando en el puerto 3000.

6.3. Cliente

```
cd frontend
npm install
npm run dev
```

Queda disponible en el puerto 5173. Las peticiones dirigidas al servidor se redirigen automáticamente, de modo que ambos procesos deben estar activos al mismo tiempo.

7. Operaciones disponibles

La interfaz expone las siguientes direcciones, todas bajo el prefijo correspondiente a la API:

- **GET /api/salud:** verificación de que el servicio responde.
- **GET /api/productos:** listado del catálogo con filtros y paginación.
- **GET /api/productos/:id:** consulta de un producto con su stock calculado.
- **POST /api/productos:** alta de un producto.
- **PUT /api/productos/:id:** modificación de un producto existente.
- **DELETE /api/productos/:id:** baja lógica de un producto.

Los parámetros admitidos en el listado son el texto de búsqueda, el identificador de categoría, la lista de etiquetas, el filtro de disponibilidad, el número de página y la cantidad de elementos por página.

8. Estado del backlog

Respecto de los elementos priorizados en la planificación:

- **PBI-01:** configuración del proyecto, base de datos y repositorios. Completado, 5 puntos.
- **PBI-02:** CRUD de catálogo de productos y etiquetas. Completado en su funcionalidad principal, 8 puntos. Resta la administración de etiquetas desde el panel.
- **PBI-03:** catálogo público con motor de búsqueda y filtros. En curso, 5 puntos. La búsqueda por texto y el filtro de disponibilidad están operativos; restan los filtros por especie, edad y tamaño en la interfaz.
- **PBI-04:** CRUD de gestión de usuarios y clientes. Pendiente, 5 puntos. Comienza en los próximos días dentro del mismo sprint.

El avance acumulado es de aproximadamente 15 de los 75 puntos estimados para el total del proyecto, lo que resulta consistente con la velocidad proyectada de 18 a 20 puntos por sprint.

9. Próximos pasos

Dentro del Sprint 2, que cierra el 11 de noviembre:

- Implementar la autenticación con token y la verificación de rol, para proteger las operaciones de alta, modificación y baja.
- Desarrollar el CRUD de usuarios y clientes.
- Agregar a la interfaz los filtros por especie, edad y tamaño basados en las etiquetas.
- Incorporar la administración de categorías y etiquetas desde el panel.

A partir del Sprint 3, que comienza el 12 de noviembre, el trabajo se concentra en la persistencia del carrito y en la integración con el entorno de prueba de la pasarela de pagos, que es el elemento de mayor riesgo del cronograma y por eso se adelantó respecto de otras funcionalidades.

10. Observaciones

No se registraron bloqueos durante esta etapa. El entorno de desarrollo quedó funcionando en las máquinas de los tres integrantes y el repositorio está operativo.

Se mantiene como principal riesgo del proyecto la integración con las interfaces externas de pago y de logística, según lo identificado en la planificación. La mitigación prevista consiste en realizar las pruebas en entornos de prueba con anticipación, y en el caso de la logística, contemplar la carga manual del seguimiento como alternativa si la interfaz del correo no responde.

También continúan pendientes de definición con los clientes algunas cuestiones registradas en el relevamiento, en particular el tratamiento de las presentaciones de un

mismo alimento como variantes o como productos independientes, que afecta la forma de contar el stock. La resolución de ese punto se necesita antes de avanzar con el módulo de carrito.